



DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

First Assignment

Equivalent representations of orientation matrices

Author:

Allegri Luca
Laura Andrea
Zerbato Antonio

Student ID:

s7905713
s4887380
s8999827

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

November 11, 2025

Contents

1	Assignment description	3
1.1	Exercise 1 - Angle-Axis to Rotation Matrix	3
1.2	Exercise 2 - Rotation Matrix to Angle-Axis	3
1.3	Exercise 3 - Euler Angles to Rotation Matrix	4
1.4	Exercise 4 - Rotation Matrix to Euler Angles	4
1.5	Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors	4
1.6	Exercise 6 - Frame tree	5
2	Exercise 1	6
2.1	Q1.1	6
2.2	Q1.2	6
2.3	Q1.3	6
2.4	Q1.4	6
3	Exercise 2	7
3.1	Q2.1	7
3.2	Q2.2	7
3.3	Q2.3	7
3.4	Q2.4	8
3.5	Q2.5	8
3.6	Q2.6	8
4	Exercise 3	8
5	Exercise 4	10
6	Exercise 5	10
7	Exercise 6	11
7.1	Q6.1	11

Mathematical expression	Definition	MATLAB expression
$\langle w \rangle$	World Coordinate Frame	w
${}^a_b R$	Rotation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aRb
${}^a_b T$	Transformation matrix of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$	aTb

Table 1: Nomenclature Table

1 Assignment description

The first assignment of Modelling and Control of Manipulators focuses on the geometric fundamentals and algorithmic tools underlying any robotics application. The concepts of transformation matrix, orientation matrix and the equivalent representations of orientation matrices (Equivalent angle-axis representation and Euler Angles) will be reviewed.

The first assignment is **mandatory** and consists of 5 different exercises. You are asked to:

- Download the .zip file called MCM-LAB1 from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the predefined files called "main.m", "AngleAxisToRot.m", "RotToAngleAxis.m", "YPRToRot.m", "RotToYPR.m" and "IsRotationMatrix.m".
- Write a report motivating the answers for each exercise, following the predefined format on this document.
- The usage of built-in MATLAB functions is strictly forbidden except for basic mathematical operations such as *det*, *eig*.

1.1 Exercise 1 - Angle-Axis to Rotation Matrix

A particularly interesting minimal representation of 3D rotation matrices is the so-called angle-axis representation, where a rotation is represented by the axis of rotation $\mathbf{h}$ and the angle θ . Any rotation matrix can be represented by its equivalent angle-axis representation by applying the Rodrigues Formula.

Q1.1 Given an angle-axis pair $(\mathbf{h}, \theta)$, implement on MATLAB the Rodrigues formula, computing the equivalent rotation matrix, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{AngleAxisToRot}(h, \theta)$$

Then test it for the following cases and briefly comment the results obtained:

- **Q1.2** $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90^\circ$
- **Q1.3** $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$
- **Q1.4** $\rho = [-\pi/3, -\pi/6, \pi/3]$;

1.2 Exercise 2 - Rotation Matrix to Angle-Axis

Given a rotation matrix R , the problem of finding the corresponding angle-axis representation $(\mathbf{h}, \theta)$ is called the Inverse Equivalent Angle-Axis Problem.

Q2.1 Given a rotation matrix R , implement on MATLAB the Equivalent Angle-Axis equations **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [h, \theta] = \text{RotToAngleAxis}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix*(R).

Hint: utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q2.2** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$
- **Q2.3** $R = \begin{pmatrix} 0.5 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.4** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.5** $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

• **Q2.6**
$$R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

1.3 Exercise 3 - Euler Angles to Rotation Matrix

Any orientation matrix can be expressed in terms of three elementary rotations in sequence. Consider the Yaw Pitch Roll (YPR) representation, where the sequence of the rotation axes is Z-Y-X.

Q3.1 Given a triplet of YPR angles (ψ, θ, ϕ) , compute the equivalent rotation matrix representation **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } R = \text{YPRToRot}(\text{psi}, \text{theta}, \text{phi})$$

Then test it for the following cases and briefly comment the results obtained:

- **Q3.2** $\psi = \theta = 0, \phi = \pi/2$
- **Q3.3** $\phi = \theta = 0, \psi = 60^\circ$
- **Q3.4** $\psi = \pi/3, \theta = \pi/2, \phi = \pi/4$
- **Q3.5** $\psi = 0, \theta = \pi/2, \phi = -\pi/12$

1.4 Exercise 4 - Rotation Matrix to Euler Angles

Given a rotation matrix R , it is possible to compute an equivalent triplet of YPR angles (ψ, θ, ϕ) , provided that the configuration is not singular (that is, $\cos \theta \neq 0$).

Q4.1 Given a rotation matrix R , implement in MATLAB the equivalent YPR angles, **WITHOUT** using built-in matlab functions. The function signature will be

$$\text{function } [\text{psi}, \text{theta}, \text{phi}] = \text{RotToYPR}(R)$$

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix*(R). **Hint:** utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.

Test it for the following cases and briefly comment the results obtained:

- **Q4.2**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$
- **Q4.3**
$$R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$
- **Q4.4**
$$R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$$

1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors

Consider a rotation matrix R and the corresponding angle-axis representation $(\mathbf{h}, \theta)$. Then, $R\mathbf{h} = \mathbf{h}$, which can be verified by computing $R\mathbf{h}$ with the Rodrigues Formula and observing that the terms containing $\mathbf{h} \times \mathbf{h}$ vanish. Therefore, $\mathbf{h}$ is an eigenvector of R corresponding to the eigenvalue +1.

Given the following rotation matrices, compute the corresponding angle-axis representation with two different methods. In the first method, compute $(\mathbf{h}, \theta)$ using the function *RotToAngleAxis* implemented before. In the second one, compute $\mathbf{h}$ as the eigenvector of R corresponding to eigenvalue +1, and then use the function *RotToAngleAxis* to compute θ . Compare and briefly discuss the results.

• **Q5.1**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

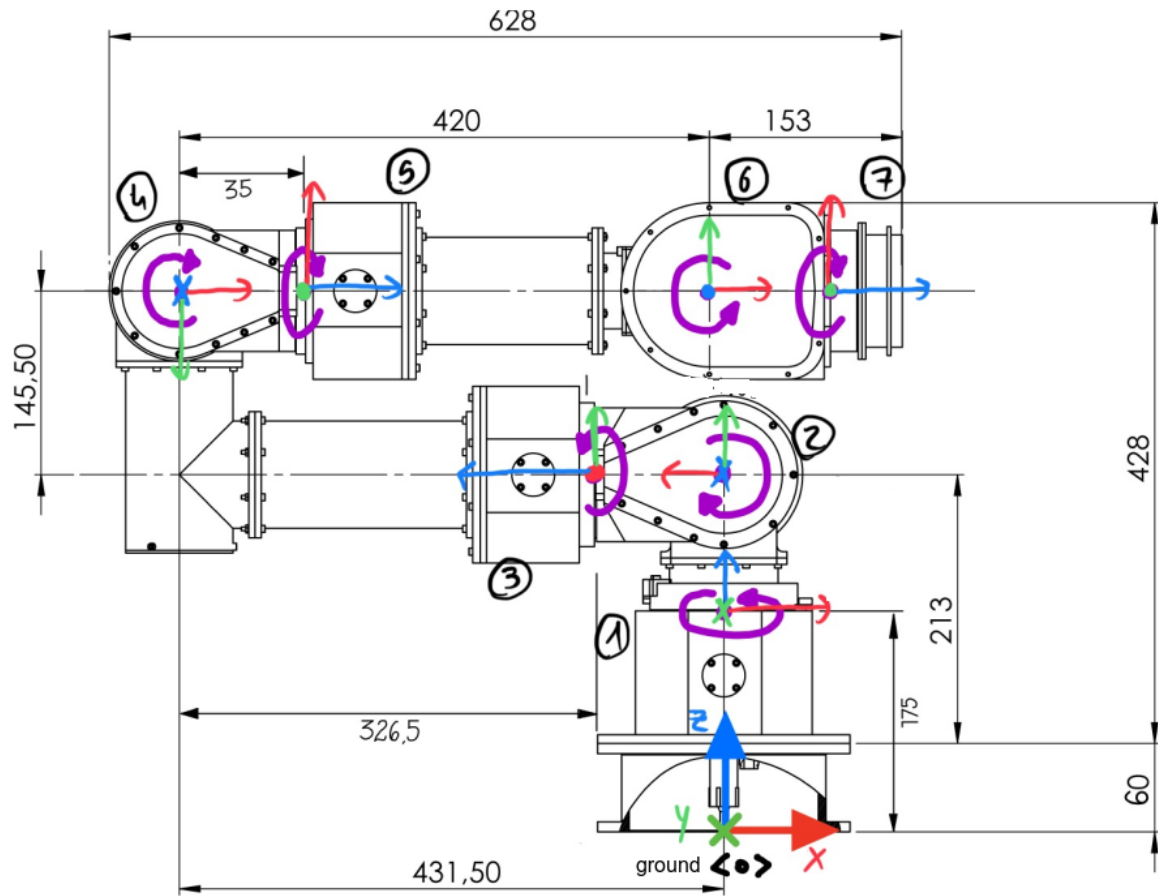


Figure 1: Exercise 6 frames. The unit of measurement is millimeters.

• Q5.2
$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

1.6 Exercise 6 - Frame tree

Figure 1 shows the frame tree for the 7 joints of the Franka robot. With reference to the figure, use the geometric definition of the transformation matrix to compute by hand the following matrices.

- Q6.1 0_1T
- Q6.2 1_2T
- Q6.3 2_3T
- Q6.4 3_4T
- Q6.5 4_5T
- Q6.6 5_6T
- Q6.7 6_7T
- Q6.8 7_6T

You **MUST** compute the matrices **WITHOUT** using mathematical software.

2 Exercise 1

2.1 Q1.1

The first exercise consists of transforming the angle-axis representation into its corresponding rotation matrix. For the angle-axis formulation we need a unit vector $\mathbf{h}$, which defines the axis of rotation, and an angle θ , which specifies the magnitude of the rotation. To convert an angle-axis representation into its corresponding rotation matrix R we apply the Rodrigues Formula. This formula incorporates the identity matrix $\mathbf{I}$, an anti-symmetrical part which is $\sin(\theta) \cdot \mathbf{h}$ and a symmetrical part $(1 - \cos(\theta)) \cdot \mathbf{h}^2$.

$$R = I + \sin(\theta) \cdot \mathbf{h} + (1 - \cos(\theta)) \cdot \mathbf{h}^2$$

2.2 Q1.2

$$\mathbf{h} = [1 \ 0 \ 0]^T \quad \theta = 90^\circ$$

Using the Rodrigues formula, we obtain a rotation matrix with the x-axis unchanged while the y-axis and z-axis rotating by 90 within the yz-plane. This is a standard case where a matrix rotates about the x-axis.

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

2.3 Q1.3

$$\mathbf{h} = [0 \ 0 \ 1]^T \quad \theta = \pi/3$$

Given the unit vector $\mathbf{h}$ aligned with the z-axis, the Rodrigues formula generates a rotation matrix that keeps the z-component unchanged while rotating the x- and y-components by $\pi/3$ in the xy-plane. The resulting matrix corresponds to a standard planar rotation.

$$R = \begin{bmatrix} 1/2 & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & 1/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2.4 Q1.4

$$\boldsymbol{\rho} = \left[-\frac{\pi}{3} \quad -\frac{\pi}{6} \quad \frac{\pi}{3}\right]$$

Given the rotation vector $\boldsymbol{\rho}$, which by definition is expressed as $\boldsymbol{\rho} = \theta \mathbf{h}$, where the direction of $\mathbf{h}$ specifies the axis of rotation and the norm of $\boldsymbol{\rho}$ corresponds to the rotation angle θ , we proceed as follows. The rotation angle is computed as:

$$\theta = \|\boldsymbol{\rho}\| = \sqrt{\left(-\frac{\pi}{3}\right)^2 + \left(-\frac{\pi}{6}\right)^2 + \left(\frac{\pi}{3}\right)^2} = \frac{\pi}{2}$$

The corresponding unit vector $\mathbf{h}$, which defines the axis of rotation, is given by:

$$\mathbf{h} = \frac{\boldsymbol{\rho}}{\theta} = \begin{bmatrix} h_x \\ h_y \\ h_z \end{bmatrix}$$

Subsequently, we construct the skew-symmetric matrix associated with $\mathbf{h}$, and apply the Rodrigues rotation formula to obtain the rotation matrix. The resulting rotation is characterized by an axis:

$$\mathbf{h} = \begin{bmatrix} -0.6667 \\ -0.3333 \\ 0.6667 \end{bmatrix}$$

and a rotation angle $\theta \approx 1.5708$ radians. The final rotation matrix R is:

$$R = \begin{bmatrix} 0.4444 & -0.4444 & -0.7778 \\ 0.8889 & 0.1111 & 0.4444 \\ -0.1111 & -0.8889 & 0.4444 \end{bmatrix}$$

3 Exercise 2

3.1 Q2.1

This exercise focuses on the Inverse Equivalent Angle-Axis Problem, which transforms a rotation matrix R into its corresponding angle-axis representation (h, θ) . The first step done in the *RotToAngleAxis* function is to verify if the input matrix R is a valid rotation matrix using the `IsRotationMatrix` function, which checks if the matrix is orthogonal ($R^T R = I$) and its determinant is equal to 1 ($\det(R) = 1$). Once the validity of the rotation matrix is confirmed, we can compute the rotation angle θ using the formula:

$$\theta = \cos^{-1} \left(\frac{\text{trace}(R) - 1}{2} \right)$$

The computation of the rotation axis h may admit multiple solutions depending on the value of the rotation angle θ :

- if $\theta \approx 0$, we can assume an arbitrary rotation axis h since there is no rotation around it;
- if $0 < \theta < \pi$, h is proportional to the antisymmetric part of R . Therefore, to find h , the axial vector of $\frac{R - R^T}{2}$ is first calculated with the `vex` function, then divided by $\sin(\theta)$, and finally normalized to ensure a unit norm.
- if $\theta \approx \pi$, we are in a special case. Using the previous formula, we would end up in an indeterminate case ($\sin \pi = 0$). The Rodrigues Formula is reduced to $R = I + 2[h]^2 = I + 2(hh^T - I) = 2hh^T - I$. At this point, we compute one of the components of the h vector using the formula

$$h_i = \pm \sqrt{\frac{R_{i,i} + 1}{2}}$$

where i could be x,y or z. Because the values of the remaining two components are determined by the sign of the first, it is necessary to select the index i such that h_i is nonzero. Given the sign ambiguity introduced by the square root, for convenience, we assume the calculated component h_i has a positive sign. Once the first nonzero component h_i has been determined, the remaining components can be obtained from:

$$h_j = \text{sign}(h_i) \text{sign}(R_{ij}) \sqrt{\frac{R_{ji} + 1}{2}}, \quad j \neq i.$$

3.2 Q2.2

Given the Rotation matrix:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

The rotation angle θ is computed using the trace of R and the result is $\theta = \frac{\pi}{2} = 1.5708$ rad. We are in the second case of above ($0 < \theta < \pi$) so we can use the generic formula to compute h that results: $\mathbf{h} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$.

3.3 Q2.3

Given the rotation matrix:

$$R = \begin{bmatrix} 0.5 & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

As point **Q2.3** we are in the case ($0 < \theta < \pi$) because $\theta = \frac{\pi}{3}$. $\mathbf{h} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, so this matrix represents a rotation of 60° around the z -axis.

3.4 Q2.4

Given the rotation matrix:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

With this rotational matrix, θ is approximate equal to 0, and as said before, there are no rotation around h and we can assume it in an arbitrary way, such as $\mathbf{h} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$.

3.5 Q2.5

Given the rotation matrix:

$$R = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Computing θ we realize we are in the third case where $\theta \approx \pi$. $\mathbf{h} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, it means there is a rotation of 180° around the z -axis, so z -axis remains invariant and the x and y components are flipped in direction.

3.6 Q2.6

Given the rotation matrix:

$$R = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This last matrix does not satisfy all the condition within the `IsRotationMatrix` function. Although the orthogonality condition $R^T R = I$ is satisfied, the determinant of R is -1 , which means it is not a valid rotation matrix.

4 Exercise 3

In the third exercise we implemented how, given three intrinsic rotations along the rotation axes, we obtain the corresponding rotation matrix. The provided angles, defined as ϕ, θ and ϕ , correspond respectively to rotations about the z -axis, the new y -axis (after the z -rotation), and finally the new x -axis (after the previous two rotations). After that, the overall rotation matrix R can be expressed as the ordered product of the individual rotation matrices:

$$R = R_Z(\psi)R_Y(\theta)R_X(\phi) \quad (1)$$

Where:

$$R_Z(\psi) = \begin{pmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} R_Y(\theta) = \begin{pmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{pmatrix} R_X(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c\phi & -s\phi \\ 0 & s\phi & c\phi \end{pmatrix}$$

So, the final matrix R , expressed in terms of cosines c and sines s of the rotation angles, is given by:

$$R = \begin{pmatrix} c\psi c\theta & -s\psi c\phi + c\psi s\theta s\phi & s\psi s\phi + c\psi c\phi s\theta \\ s\psi c\theta & c\psi c\phi + s\psi s\theta s\phi & -c\psi s\phi + s\theta s\psi c\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{pmatrix} \quad (2)$$

Hence, the `YPRTtoRot` function takes as input the three rotation angles ϕ, θ , and ϕ , computes the fundamental rotation matrices $R_Z(\psi)$, $R_Y(\theta)$, and $R_X(\phi)$ and subsequently returns the resulting composite rotation matrix R , obtained from their ordered product.

- **Q3.2** $\psi = \theta = 0, \phi = \pi/2$

A rotation of 90° is applied about the x-axis, with no additional rotations. As a result, the final rotation matrix can be interpreted as follows: the x-axis remains unchanged, the y-axis is aligned with the original z-axis, and the z-axis is oriented in the opposite direction to the original y-axis

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

- **Q3.3** $\phi = \theta = 0, \psi = 60^\circ$

In this case, a pure rotation of 60° is applied about the Z-axis. Consequently, the X- and Y-axes are reoriented with respect to their original positions, whereas the Z-axis remains invariant:

$$R = \begin{pmatrix} 0.5 & -0.8660 & 0 \\ 0.8660 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- **Q3.4** $\psi = \pi/3, \theta = \pi/2, \phi = \pi/4$

For these particular rotation angles, a complete rotation matrix is given:

$$R = \begin{pmatrix} 0 & -0.2588 & 0.9659 \\ 0 & 0.9659 & 0.2588 \\ -1 & 0 & 0 \end{pmatrix}$$

- **Q3.5** $\psi = 0, \theta = \pi/2, \phi = -\pi/12$

Also for this example, we obtained a complete rotation matrix:

$$R = \begin{pmatrix} 0 & -0.2588 & 0.9659 \\ 0 & 0.9659 & 0.2588 \\ -1 & 0 & 0 \end{pmatrix}$$

It should be noted that in points **Q3.4** and **Q3.5**, although the ϕ and ψ angles provided are different, the resulting rotation matrix remains the same. This is because in both cases $\theta = \pi/2$, which corresponds to a singularity known as "Gimbal Lock" for the Yaw-Pitch-Roll rotation sequence.

When the central pitch angle is $\pm 90^\circ$, the axis of the first rotation around z becomes aligned with the axis of the final rotation (about x) after the rotation about y . This alignment triggers the Gimbal Lock condition.

This results in a loss of a degree of freedom, and on the ability to distinguish between a change in ψ and a change in ϕ . In fact, for $\theta = \pm\pi/2$, $\cos\theta = 0$, and the matrix depends only on the difference between Yaw and Roll angles:

$$R(\psi, \pi/2, \phi) = \begin{pmatrix} 0 & -s\psi c\phi + c\psi s\phi & s\psi s\phi + c\psi c\phi \\ 0 & c\psi c\phi + s\psi s\phi & -c\psi s\phi + s\psi c\phi \\ -1 & 0 & 0 \end{pmatrix}$$

That for the addition/subtraction formulas of angles:

$$R(\psi, \pi/2, \phi) = \begin{pmatrix} 0 & s(\phi - \psi) & c(\phi - \psi) \\ 0 & c(\phi - \psi) & -s(\phi - \psi) \\ -1 & 0 & 0 \end{pmatrix}$$

Therefore, the two rotation matrices are equal because, in addition to having $\theta = \pi/2$, they have the same value for the difference $(\phi - \psi) = -\pi/12$

5 Exercise 4

The objective of this exercise is to determine the rotation angles ψ , θ , and ϕ given a rotation matrix. The implemented `RotToYPR` function first verifies whether the input matrix is a valid rotation matrix by using the `IsRotationMatrix` function. Subsequently, the angle θ is computed, which allows valuating whether the system is in a Gimbal Lock singularity condition, when $\theta = \pm\pi/2$.

If $\theta \neq \pm\pi/2$, and thus $\cos\theta \neq 0$, no singularity occurs and the rotation angles can be uniquely determined. Instead, when $\theta = \pm\pi/2$ the system is in a singular configuration, and the inverse mapping is not unique. In this case, one of the angles —typically ψ — is conventionally set to zero as an arbitrary choice, so that the remaining rotation is entirely attributed to the Roll angle ϕ . This approach ensures that the function returns a well-defined and consistent triplet of angles. Hence:

• **Q4.2**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

For this rotation matrix, which corresponds to the result obtained in **Q3.2**, the same Euler angles are produced by applying a rotation of 90° around the X-axis $\psi = \theta = 0$, $\phi = 90^\circ$.

• **Q4.3**
$$R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

The Euler angles that coincide with this matrix R are: $\phi = \theta = 0$, $\psi = 60^\circ$, which correspond to the angles given in exercise **Q3.3** and which led to the same rotation matrix.

• **Q4.4**
$$R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$$

Finally, in this last rotation matrix the Euler angles that represent it are: $\psi = 90^\circ$, $\theta = 60^\circ$, $\phi = 45^\circ$, it means that we have a 90° rotation around z -axis, after 60° rotation around the newest y -axis, then a rotation of 45° around the newest x -axis.

In this exercise no matrix has a corresponding θ angle of 90° and therefore there are no singularities of the Gimbal Lock type.

6 Exercise 5

The aim of this section of the assignment is to compare the results obtained with the function `RotToAngleAxis`, with those derived from the extrapolation of the vector h , considered as the eigenvector of R corresponding to the eigenvalue $+1$.

Given a generic matrix R the definition of an eigenvector v , associated with its corresponding eigenvalue λ is the following:

$$Rv = \lambda v$$

The geometrical meaning of h being the eigenvector of R with eigenvalue 1 can be explained as follows: when a rotation matrix is applied to a vector space, the transformation changes the components of all vectors except those lying along the rotation axis.

To obtain the desired result, a function named `eigen_H` was implemented. The function takes the rotation matrix R as its input parameter and returns the vector h as the output. The logic of the function works as follows: the MATLAB function `eig` was first used to compute the matrices containing the eigenvectors and eigenvalues of the given rotation matrix. Then, using a simple for loop, the index of the column corresponding to the eigenvalue 1 was identified, and finally the resulting eigenvector h was extracted from the eigenvector matrix with the previous index. A normalization of the vector was then performed to ensure its norm equaled 1.

• **Q5.1**
$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$$

From the structure of the rotation matrix, the first column shows how the rotation is computed along the x -axis. As expected, both the `RotToAngleAxis` and `eigen_H` functions produce the following h vector associated with an angle $\theta = \pi/2$:

$$h = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

• **Q5.2**
$$R = \frac{1}{9} \begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$$

From the given matrix, the function `RotToAngleAxis` returned a vector $h = [-0.6667, -0.3333, 0.6667]$ and an angle $\theta = \pi/2$.

On the other hand, the axis vector h generated by `eigen_H` was originally $h = [0.6667, 0.3333, -0.6667]$. The reason for this lies in the Rodrigues Formula. This formula, implemented inside `RotToAngleAxis`, always considers positive rotation angles, while the direction of the rotation is encoded in the sign of the h vector components. However, the `eigen_H` does not take the rotation angle into account. It can be observed that the eigenvectors associated with the eigenvalue 1 are both h and $-h$. As a matter of fact:

$$Rh = \lambda h \quad \text{but also}$$

$$R(-h) = \lambda(-h)$$

The geometrical meaning is straightforward. The rotation axis is the only line that remains unchanged during the rotation, but every line has two possible directions. To address this issue, the axis vector h_{dot} , associated with the Rodrigues formula, was computed within the `eigen_H` function, and the sign of the output eigenvector was adjusted according to its scalar product with h_{dot} . Therefore, assuming a positive rotation, from the given matrix the `eigen_H` function extracts a vector h :

$$h = \begin{pmatrix} -0.6667 \\ -0.3333 \\ 0.6667 \end{pmatrix}$$

7 Exercise 6

7.1 Q6.1

The sixth task of the assignment consisted of computing the seven joint transformation matrices of the Franka robot with reference to the given figure Fig:1. A transformation matrix is a square matrix that represents a linear transformation. It maps vectors from one coordinate system to another while preserving the structure of the space.

Each transformation matrix has a generic composition structure:

$${}^a_bT = \begin{pmatrix} {}^a_bR & {}^a_bO \\ 0_{1 \times 3} & 1 \end{pmatrix}$$

In this representation, a_bR is the Rotation matrix describing the orientation of the axis vector of frame $\langle b \rangle$ with respect to the axis vector of frame $\langle a \rangle$, a_bO is the vector that connects the origins of the two reference frames, and $0_{1 \times 3}$ is the zero vector of one row and three columns.

This construction brings several advantages to the computation of the robot kinematics, since it allows us to compute both the orientation and the translation of each frame within a single 4×4 square matrix.

Based on this information, the transformation matrices were computed starting from frame $\langle 0 \rangle$ (the ground frame). The first step was to determine the orientation matrix. The procedure is straightforward: to obtain the elements of the rotation matrix, we align the origins of the two frames and describe the components of the three axes of frame $\langle 1 \rangle$ with respect to the unit axis vectors of frame $\langle 0 \rangle$. These values are then placed in the corresponding columns.

As shown in the figure, the x, y, z -axes orientations of frame $\langle 0 \rangle$ and frame $\langle 1 \rangle$ are identical. The resulting orientation matrix R is:

$${}^0_1R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

The translation vector was then computed. The figure shows a translation toward the z -axis of 175. The unit of measurement is millimeters. Hence, the translation vector is:

$${}^0_1T = \begin{pmatrix} 0 \\ 0 \\ 175 \end{pmatrix}$$

Given the rotation matrix and the translation vector, we can finally compute the resulting transformation matrix:

$${}^0_1T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 175 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The same procedure was applied to the other matrices.

The first column of the orientation matrix from frame $\langle 1 \rangle$ to frame $\langle 2 \rangle$ shows how the x -axis of frame $\langle 2 \rangle$ is in the opposite direction of $\langle 1 \rangle$. On the other hand, the y, z components are interchanged. Observing the figure, the translation vector can be computed through the simple operation $213 + 60 - 175$ on the z -axis.

$${}^1_2T = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 98 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Considering the transformation matrix from frame $\langle 2 \rangle$ to frame $\langle 3 \rangle$, the translation vector component is $431, 50 - 326, 50 = 105$ along the x -axis of $\langle 2 \rangle$. The orientation of frame $\langle 3 \rangle$ can be easily determined. The x -axis is represented by the vector $x_3 = [0, 0, -1]$ with respect to frame $\langle 2 \rangle$, the y -axis by $y_3 = [0, 1, 0]$, and the remaining z -axis by $z_3 = [1, 0, 0]$.

$${}^2_3T = \begin{pmatrix} 0 & 0 & 1 & 105 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

To compute the transformation from frame $\langle 3 \rangle$ to frame $\langle 4 \rangle$, the same procedure described earlier was followed. By aligning the origins of the two frames and observing the orientation of their axes, the rotation matrix can be easily computed. However, in this case, the translation vector is described by two components on the y and z axes. The resulting homogeneous transformation matrix is:

$${}^3_4T = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 145,50 \\ -1 & 0 & 0 & 326,50 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Considering the transformation from frame $\langle 4 \rangle$ to frame $\langle 5 \rangle$, the image shows how the z -axis of the second frame has the same direction as the x -axis of the first one, while the y and x -axes point in the opposite direction of the z and y axes, respectively.

$${}^4_5T = \begin{pmatrix} 0 & 0 & 1 & 35 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

In the next step, the transformation between frames $\langle 5 \rangle$ and $\langle 6 \rangle$ was determined. In this case, the value of the translation vector along the z component was computed by a simple operation such as $420 - 35 = 385$. The final homogeneous transformation matrix is:

$${}^5_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 385 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Finally, the transformation from frame $\langle 6 \rangle$ to frame $\langle 7 \rangle$ was computed. This last step describes the position and orientation of the end-effector with respect to the previous frame. Although the figure does not clearly show the distance between the two frame's origins, its value was supposed to be 153.

$${}^6_7T = \begin{pmatrix} 0 & 0 & 1 & 153 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The images in Fig.2 and Fig.3 , presented respectively in two-dimensional and three-dimensional views, illustrate the reference frames associated with the joints of the robotic system. The corresponding rotation matrices and translation vectors were obtained using the *extractRT* function from the computed transformation matrices. All the reference frames were displayed using the function *plotframe*, available on the MATLAB community [1]. This function draws a 3D Cartesian coordinate system given a rotation matrix and a translation vector. The function accepts as input parameters a rotation matrix, a translation vector (scaled by a *a* factor to obtain a better plot) and custom labels such as *MatrixIndexing* or *columnmajor*, which ensure the correct interpretation of the rotation matrix.

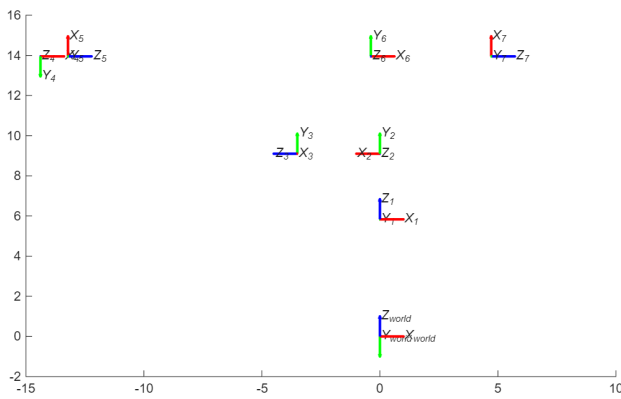


Figure 2: Plot 2D View

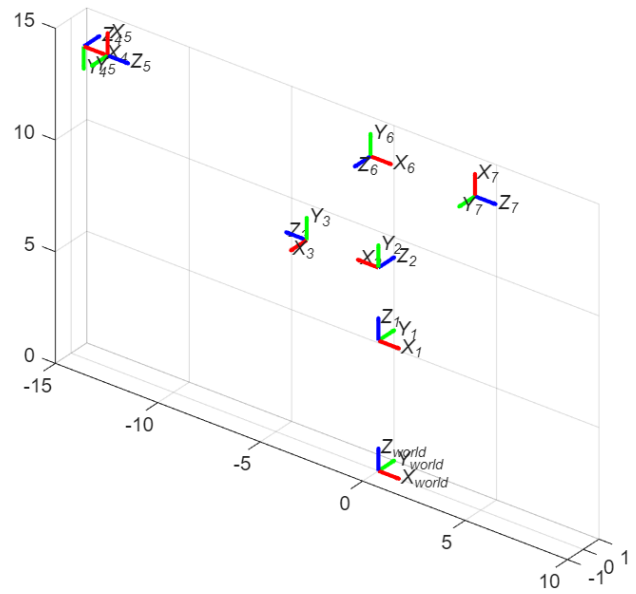


Figure 3: Plot 3D View

References

- [1] George Abrahams. *plotframe: Plot a 3-D Cartesian coordinate system*. 2023.